Tech Challenge — Fase 4

Microsserviços e Saga Pattern na Plataforma de Gestão de Oficina Mecânica

> **RASCUNHO PARA REVISÃO DO RAFAEL — NÃO ENVIAR AO PORTAL.** Itens marcados **[PENDENTE]**, **[EM ANDAMENTO]** ou **[CONFIRMAR]** não têm evidência ainda. Nenhum número de cobertura, qualidade ou observabilidade foi medido para este documento. Prazo de entrega: 17/11/2026.

Integrantes: [CONFIRMAR] nome completo e RM de cada integrante (Fase 3: Rafael Xavier · Silas Furini).

Vídeo de demonstração (até 15 min): [PENDENTE] `<link YouTube/Vimeo>`. O vídeo será gravado somente no ambiente online da AWS Academy (Learner Lab), não no Kind.

Repositórios (um por microsserviço):

Serviço	Repositório
OS Service (orquestrador da saga)	https://github.com/rafaelxvr/Tech-challenge-15SOAT
Billing Service (orçamento e pagamento)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-billing
Execution Service (diagnóstico e reparo)	https://github.com/rafaelxvr/Tech-challenge-15SOAT-execution

Revisor: [CONFIRMAR] acesso do usuário `soat-architecture` nos três repositórios (na Fase 3 constava nos quatro daquela fase; não verificado para a Fase 4).

Sumário

1. Resumo executivo
2. Arquitetura dos microsserviços
3. Saga orquestrada
4. Divisão de responsabilidades e tecnologias
5. CI/CD, Kind local e deploy na AWS Academy
6. Observabilidade (New Relic)
7. Qualidade e testes (SonarCloud)
8. Pagamento com Mercado Pago (sandbox)
9. Matriz de rastreabilidade do enunciado
10. Limitações e riscos conhecidos

1. Resumo executivo

A Fase 3 colocou a aplicação da oficina em nuvem como um serviço único. A Fase 4 a divide em **três microsserviços independentes**, cada um com repositório, pipeline, infraestrutura e banco próprios, e coordena as transações que atravessam os três (abertura da OS, orçamento, aprovação, pagamento, reserva de peças e reparo) com uma **saga orquestrada**, com compensação em caso de falha.

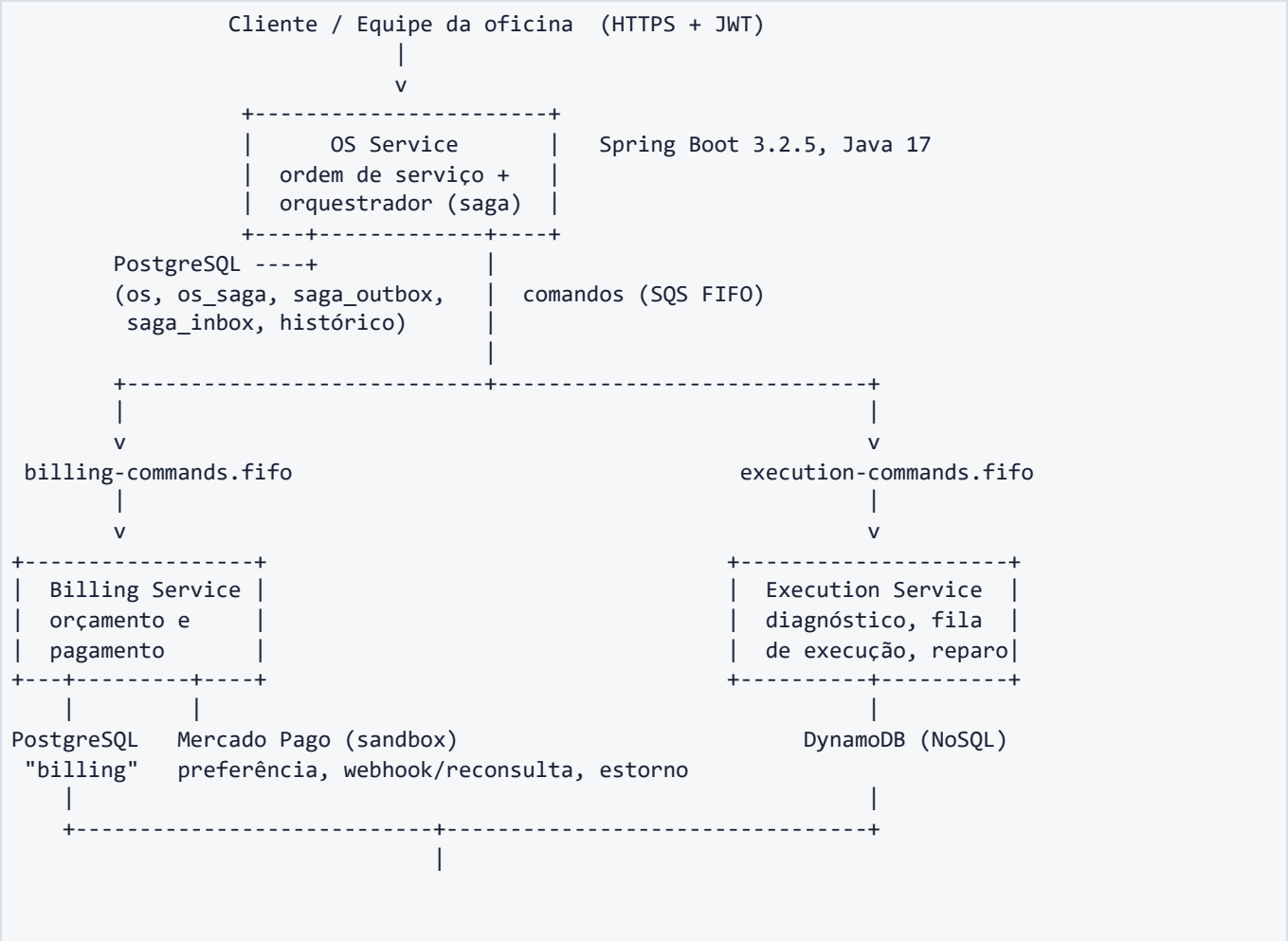
- **OS Service** é dono da ordem de serviço e orquestra a saga. Banco relacional PostgreSQL.
- **Billing Service** é dono do orçamento e do pagamento (Mercado Pago). Banco relacional PostgreSQL próprio.
- **Execution Service** é dono da fila de execução, do diagnóstico e do reparo. Banco não relacional DynamoDB.

Os serviços não acessam o banco uns dos outros. A integração é assíncrona, por filas SQS FIFO, com contratos JSON versionados.

Situação em 03/11/2026: o código dos três serviços e a saga estão em `develop` dos respectivos repositórios; a integração local usa PostgreSQL, LocalStack e Kind efêmero. O deploy na AWS Academy, a observabilidade no New Relic, a análise no SonarCloud, a prova de pagamento no sandbox do Mercado Pago e o vídeo estão **pendentes** (seções 5 a 8).

2. Arquitetura dos microsserviços

2.1 Diagrama geral



```
                                  v   respostas (SQS FIFO)
                          os-saga-replies.fifo  ----->  OS Service
```

Diagramas Mermaid versionados: `docs/architecture/c4-*.md` no repositório do OS Service e o diagrama do README do Billing Service. **[PENDENTE]** versão final do diagrama geral com os nomes dos recursos efetivamente criados na AWS Academy.

2.2 Bancos isolados

Serviço	Banco	Tipo	Justificativa
OS Service	PostgreSQL (schema com Flyway)	SQL	Agregado transacional da OS, histórico e tabelas da saga precisam de atomicidade entre linhas
Billing Service	PostgreSQL próprio (`billing`)	SQL	Orçamento, pagamento, inbox e outbox na mesma transação; conferência de valor e moeda
Execution Service	DynamoDB	NoSQL	Fila de execução e estado por OS com acesso por chave e escritas condicionais atômicas

Nenhum serviço lê ou escreve no banco de outro. Dados de outro domínio só chegam por mensagem.

2.3 Mensageria e contratos

Fila	Dona (consumidor)	Conteúdo
`billing-commands.fifo`	Billing	`GerarOrcamento`, `RegistrarDecisao`, `CancelarOrcamento`, `EstornarPagamento`
`execution-commands.fifo`	Execution	`IniciarDiagnostico`, `IniciarReparo`, `CancelarExecucao`
`os-saga-replies.fifo`	OS Service	respostas de Billing e Execution

- `MessageGroupId = osId`: ordem preservada por OS. `MessageDeduplicationId = messageId`.
- Cada resposta copia `sagaId`, `osId`, `passo`, `correlationId` e `traceparent` do comando e usa `causationId` igual ao `messageId` do comando respondido.
- Contratos são JSON Schemas versionados (`contracts/<tipo>/v<N>.json`), congelados no gate G1.5 (Billing: manifesto `CONTRACTS.sha256`).
- Entrega transacional: inbox idempotente e outbox gravados no mesmo commit do caso de uso. Fila de mensagens mortas (DLQ) após 5 recebimentos.
- REST síncrono: cada serviço expõe sua API HTTP própria (`/billing`, `/execucao`, API do OS). Swagger/OpenAPI no OS e no Billing; coleção Postman em `docs/Oficina-Mecanica.postman_collection.json` **[PENDENTE]** atualização para a Fase 4 (UNR-605).

3. Saga orquestrada

3.1 Escolha e justificativa

Estratégia: orquestração, com o orquestrador dentro do OS Service (`OsSaga`, tabela `os_saga`). Decisão registrada na ADR 006 (`docs/adrs/006-orchestrated-saga.md`).

- Um único lugar guarda estado, etapa atual, prazos e o log de compensação. A falha fica legível, testável e demonstrável (`GET /ordens-servico/{id}/saga`).
- O ciclo de vida da OS já vivia no OS Service; o orquestrador fica ao lado do agregado que ele conduz.
- Reaproveita o outbox e o SQS FIFO da Fase 3, sem broker novo.
- Coreografia foi rejeitada: a lógica de compensação se espalharia por três repositórios e ciclos de eventos são fáceis de introduzir. RabbitMQ/Amazon MQ foi rejeitado: infraestrutura nova, sem reaproveitar o outbox, custo extra.
- Custo aceito: o OS Service conhece a ordem das etapas (acoplamento do orquestrador).

3.2 Fluxo feliz

Estados: `AGUARDANDO_DIAGNOSTICO` → `AGUARDANDO_ORCAMENTO` → `AGUARDANDO_DECISAO` → `AGUARDANDO_PAGAMENTO` → `RESERVANDO_PECAS` → `AGUARDANDO_REPARO` → `EM_EXECUCAO` → `CONCLUIDA`.

#	Etapa	Comando → destino	Resposta de sucesso
1	Diagnóstico	`IniciarDiagnostico` → Execution	`DiagnosticoConcluido`
2	Orçamento	`GerarOrcamento` → Billing	`OrcamentoGerado`
3	Decisão do cliente	`RegistrarDecisao` → Billing	`OrcamentoAprovado`
4	Pagamento	(Mercado Pago)	`PagamentoConfirmado`
5	Reserva de peças	local no OS Service	—
6	Reparo	`IniciarReparo` → Execution	`ReparoIniciado`
7	Execução	—	`ExecucaoFinalizada`

3.3 Compensações

A compensação roda em ordem inversa, a partir de uma pilha persistida: cancelar execução, repor estoque (local), estornar pagamento, cancelar orçamento e, por fim, cancelar a OS. **Iniciar o reparo é o ponto de não retorno (pivot):** nada depois dele é compensado.

Falha	Motivo registrado	O que é compensado
Estoque insuficiente na reserva	`ESTOQUE_INSUFICIENTE`	estornar pagamento, cancelar orçamento, cancelar a OS
Execution recusa o reparo	`REPARO_RECUSADO`	repor estoque, estornar pagamento, cancelar orçamento, cancelar a OS
Pagamento recusado	`PAGAMENTO_RECUSADO` (e `PAGAMENTO_EXPIRADO`)	cancelar orçamento, cancelar a OS

Outras falhas previstas: `DIAGNOSTICO_RECUSADO`, `ORCAMENTO_FALHOU`, `DECISAO_EXPIRADA` e as variantes `*_SEM_RESPOSTA` (prazo vencido).

[CONFIRMAR] a coluna "O que é compensado" contra o código antes do envio: a tabela acima foi derivada da ordem da pilha da ADR 006 e dos enums do domínio, não de uma execução registrada.

3.4 Garantias de consistência

- Efeitos (comandos, ações locais), linha da saga e log de passos são gravados na **mesma transação**.
- Respostas são processadas uma vez (`saga_inbox`) e só aceitas quando `causationId` é igual ao comando aguardado; resposta atrasada ou duplicada não tem efeito.
- Toda espera tem prazo; um poller transforma espera vencida em falha e reenvia compensações não confirmadas, até 5 envios no total, antes de `FALHA_COMPENSACAO` (alerta, resolvida por runbook).
- Um pagamento aprovado **depois** do orçamento cancelado ou do pagamento expirado é estornado automaticamente no Billing.
- OS criadas antes da saga (sem linha em `os_saga`) seguem o fluxo manual da Fase 3.

3.5 Evidência da saga

Evidência	Situação
Fluxo completo e compensações testados com BDD	**[PENDENTE]** anexar execução e link (UNR-295, UNR-296 em andamento)
Execução no Kind local (smoke)	**[PENDENTE]** anexar saída do CI; smoke do Kind mesclado em `develop` (PR #84)
Execução na AWS Academy	**[PENDENTE]** depende da seção 5.3

4. Divisão de responsabilidades e tecnologias

4.1 Por que esta divisão

Serviço	Responsabilidade	Por que separado
OS Service	Abertura, status, histórico e consulta da OS; orquestração da saga	Dono do agregado central e do estado da saga
Billing Service	Orçamento (geração, envio, decisão); pagamento e estorno no Mercado Pago	Integração externa sensível, com segredos e webhook próprios; ritmo de mudança diferente
Execution Service	Fila de execução, diagnóstico e reparo; avisa a conclusão ao OS Service	Carga orientada a fila e acesso por chave, que casa com NoSQL

A divisão segue os contextos delimitados (atendimento, cobrança, execução) citados no enunciado.

4.2 Tecnologias

Item	OS Service	Billing Service	Execution Service
Linguagem e framework	Java 17, Spring Boot 3.2.5	Java 17, Spring Boot 3.5.16	Java 17, Spring Boot 3.2.5
Banco	PostgreSQL 16	PostgreSQL	DynamoDB
Mensageria	SQS FIFO	SQS FIFO	SQS FIFO
Contexto HTTP	API do OS	`/billing`	`/execucao`
Saúde	Actuator	`/billing/actuator/health`	`/execucao/actuator/health`
Imagem e manifestos	Docker, Kubernetes	Docker, Kubernetes	Docker, Kubernetes

[CONFIRMAR] versões exatas contra os `pom.xml` de `develop` no momento do envio.

5. CI/CD, Kind local e deploy na AWS Academy

5.1 Pipelines

Cada repositório tem pipeline próprio (build, testes, verificação de qualidade) e branch `main` protegida, com Pull Request obrigatório. Fluxo: `feature` → PR para `develop` → `develop` integra → `main` contém o entregue.

Item	Situação
Pipeline de build e testes nos três repositórios	Implementado em `develop` **[CONFIRMAR]** com link da última execução verde de cada um
Proteção de `main` com PR e checagens obrigatórias	**[CONFIRMAR]** configuração em cada repositório
Etapa de qualidade (SonarCloud) no CI	**[PENDENTE]** (seção 7)
Deploy automatizado em Kubernetes	Kind: ver 5.2. AWS Academy: ver 5.3

5.2 Kind local (ambiente de teste)

Tudo é testado localmente antes de qualquer deploy: PostgreSQL, LocalStack (SQS/DynamoDB, com credenciais fictícias) e cluster Kind efêmero. Nenhum recurso AWS é usado nessa etapa. Smoke do Kind está em `develop` do OS Service (PR #84). **[PENDENTE]** evidência de execução fim a fim dos três serviços no Kind.

5.3 Deploy na AWS Academy (Learner Lab) — [EM ANDAMENTO]

O deploy na AWS Academy é requisito da entrega e **ainda não foi feito**. Restrições do ambiente que condicionam o plano:

- IAM: não é possível criar roles; usar `LabRole`, `LabInstanceProfile` e `LabEksClusterRole`.
- Instâncias de nano a large, no máximo 9 instâncias e 32 vCPU por região.
- Orçamento limitado: `terraform destroy` ao fim de cada sessão.
- Credenciais expiram com a sessão do laboratório (cerca de 4 horas).
- Toda ação na conta depende de autorização do Rafael, por recurso.

[PENDENTE] arquitetura de implantação efetiva, endereços e evidência do deploy automatizado com validação de testes.

6. Observabilidade (New Relic)

[PENDENTE] Esta seção não afirma nada medido. A Fase 3 já usa New Relic; a Fase 4 deve estender para os três serviços. A conta e as chaves são fornecidas pelo Rafael fora de banda.

Item exigido no vídeo	Situação
Monitoramento dos três serviços (latência, CPU, memória, saúde)	**[PENDENTE]**
Rastreamento distribuído entre serviços (`traceparent`, `correlationId` já trafegam nas mensagens)	**[PENDENTE]** confirmar propagação ponta a ponta no New Relic
Alerta para saga em `FALHA_COMPENSACAO`	**[PENDENTE]**
Dashboard de saga (volume, duração, falhas)	**[PENDENTE]**

7. Qualidade e testes (SonarCloud)

[PENDENTE] Nenhum percentual de cobertura ou resultado de Quality Gate foi medido para este documento.

Requisito	Situação
Testes unitários nos três serviços	**[PENDENTE]** anexar resultado por serviço
Pelo menos um fluxo completo com BDD	**[PENDENTE]** (seção 3.5)
Cobertura mínima de 80% por serviço	**[PENDENTE]** valor medido por serviço (JaCoCo/SonarCloud)
Verificação de qualidade no CI (SonarCloud)	**[PENDENTE]** conta e chaves fornecidas pelo Rafael

8. Pagamento com Mercado Pago (sandbox)

O Billing Service integra o Mercado Pago por uma camada anticorrupção (`GatewayPagamento`): preferência de checkout, consulta de pagamento, busca por `external_reference` e estorno. A notificação do Mercado Pago nunca é confiada: o pagamento é relido no provedor e `external_reference`, valor e moeda (BRL) são conferidos antes de confirmar; divergência não confirma e estorna.

- Webhook `POST /webhooks/mercadopago` com assinatura HMAC, janela de tempo, deduplicação e limites de corpo e taxa.
- Sem túnel HTTPS aprovado, o webhook real não alcança o Kind; a alternativa suportada é a reconsulta (`POST /pagamentos/{id}/reconsulta`), que usa o mesmo caminho verificado.

- O gateway fake existe só no perfil de testes e **não** é evidência de pagamento real.
- **[PENDENTE]** Prova no sandbox, executada pelo Rafael via Postman quando a coleção estiver pronta (UNR-605). Segredos (`MP_ACCESS_TOKEN`, `MP_WEBHOOK_SECRET`) são injetados por ambiente e nunca versionados.

9. Matriz de rastreabilidade do enunciado

Requisito	Evidência	Situação
Mínimo de 3 microsserviços, cada um com repositório, infraestrutura e banco	Seções 2 e 4	Implementado em `develop`
SQL e NoSQL	PostgreSQL (OS, Billing) e DynamoDB (Execution), seção 2.2	Implementado em `develop`
Nenhum serviço acessa o banco de outro	Seção 2.2	Por desenho; **[CONFIRMAR]** teste ou grants
Comunicação REST e mensageria	Seção 2.3	Implementado em `develop`
Saga com rollback e compensação, justificada	Seção 3, ADR 006	Implementado; execução **[PENDENTE]**
Integração Mercado Pago	Seção 8	Código pronto; prova sandbox **[PENDENTE]**
Testes unitários e BDD	Seção 7	**[PENDENTE]**
Cobertura mínima de 80% por serviço	Seção 7	**[PENDENTE]**
Qualidade no CI (SonarCloud)	Seção 7	**[PENDENTE]**
Pipelines independentes e `main` protegida	Seção 5.1	**[CONFIRMAR]**
Deploy automatizado em Kubernetes	Seção 5	Kind **[PENDENTE]** evidência; AWS Academy **[EM ANDAMENTO]**
Observabilidade da Fase 3 reutilizada	Seção 6	**[PENDENTE]**
Swagger ou coleção Postman atualizada	Seção 2.3	**[PENDENTE]**
Vídeo de até 15 min	Capa	**[PENDENTE]** (AWS Academy)
PDF com participantes, links, diagrama, saga e justificativas	Este documento	Rascunho

10. Limitações e riscos conhecidos

- **Prazo:** 17/11/2026. O deploy na AWS Academy, o New Relic, o SonarCloud, o BDD e a prova do Mercado Pago ainda precisam ser concluídos e evidenciados.
- **Learner Lab:** credenciais de cerca de 4 horas e orçamento limitado exigem ciclos curtos de provisionamento e `terraform destroy`; o vídeo depende de o ambiente estar no ar durante a gravação.
- **Webhook Mercado Pago:** sem túnel HTTPS, a confirmação usa reconsulta, não a notificação real.
- **Compensação:** `FALHA_COMPENSACAO` após 5 envios exige intervenção por runbook; não é automática.
- Este documento não declara cobertura, Quality Gate, latência ou disponibilidade; qualquer número deve vir de medição registrada.